



为每一位前端开发者重塑AI时代下的核心竞争力

# 企业级类剪映视频智能剪辑工具架构设计与实践 项目介绍 (Vibe Coding + FFmpeg + Langchainjs + Electron + Vue3) 项目介绍

## 需求背景介绍

在内容创作爆发的今天，视频已经成为互联网最重要的信息载体之一，短视频、知识付费、企业营销、个人记录等场景的视频需求量持续高速增长。然而，传统的专业视频剪辑工具（如 Premiere Pro、Final Cut Pro）学习曲线陡峭，操作复杂度高，对于非专业剪辑师来说门槛极高；即使是专业创作者，大量重复性的剪辑工作也会消耗大量时间，真正用于创意构思的精力反而被挤占。

**miaoma-magicut（妙码智能剪辑）**正是针对这一行业痛点打造的 **AI 驱动桌面端智能剪辑平台**，是一款**垂类专业化的音视频剪辑 Agent**。它的核心定位是：让大模型能力深度适配剪辑行业 workflow，从创意输入到成片输出全链路智能化，通过多节点智能体编排自动完成大部分重复性工作，只把创意决策留给人类。

## 核心目标用户分层

- 内容创作者**：短视频博主、知识付费讲师、自媒体从业者。他们的核心痛点是产量要求高、重复性剪辑工作多，希望通过 AI 提升产出效率，把精力集中在内容创意本身。

2. **企业营销人员**：市场、运营、培训岗位员工，需要制作产品宣传片、培训视频、活动记录视频。他们的核心痛点是不专业、剪辑能力弱，希望降低制作门槛，快速产出可用的视频。
3. **普通个人用户**：用于旅行记录、家庭相册、生活分享。他们的核心痛点是完全不会专业剪辑，希望通过简单操作就能生成有质感的视频。

## 垂类智能体的业务与技术价值

传统 AI 剪辑工具大多只是在某个单点（比如字幕生成、模板套用）做 AI 增强，没有从全链路重构创作流程。本项目作为**完整的音视频剪辑 Agent**，核心价值体现在：

### 业务价值

- **端到端自动化**：用户只需要输入自然语言创意，Agent 自动完成从创意简报 → 分镜规划 → 素材匹配 → 语音合成 → 时间线组装全流程，把创作周期从「小时级」压缩到「分钟级」
- **人机协同最优分工**：Agent 承担 80% 重复性劳动（分镜拆解、素材匹配、语音合成、时间线对齐），人类只负责最后的 20% 创意调整和质量确认，真正做到「AI 提效 + 人类创意」
- **本地隐私保护**：所有素材和创作过程都在用户本地完成，不需要上传敏感内容到云端，满足隐私合规要求
- **可编辑可调整**：AI 生成不是终点，而是起点，用户随时可以在可视化编辑器中做精细化调整，兼顾自动化效率和创作灵活性

### 技术价值

- 验证了 **LangGraph 垂类业务落地**：在真实的音视频创作场景中，实践多步骤、有状态、需要人机交互的复杂智能体设计
- 沉淀了 **混合云 AI 架构最佳实践**：在线大模型保证效果，本地模型满足隐私和离线需求，通过适配器模式做到无缝切换和故障降级
- 完整工程化链路：覆盖从需求、架构、编码、联调、测试到打包签名分发全流程，是学习 AI 桌面应用开发的绝佳实战项目

## 产品核心价值主张

- **降门槛**：不用学习复杂的剪辑软件，用自然语言就能生成视频
- **提效率**：重复性剪辑工作 AI 自动完成，制作周期从小时级缩短到分钟级
- **保质量**：内置专业剪辑规范与最佳实践，AI 生成的结果达到基础专业水准

# 学习成果

## 需求分析与架构设计

- **深入理解 AI 开发方法论体系：**系统掌握 Vibe Coding 迭代式开发、Spec 驱动的规格化研发、Skills 技能体系化赋能、MCP 工具互联协议等 AI 时代核心开发范式，清晰理解每种方法的核心思想、落地原则与适用边界，能够准确判断不同场景下的最优开发模式，避免方法错配导致的效率损耗
- **掌握产品需求分析与技术评审方法论：**系统掌握工具类产品的需求拆解与分析方法，能够从真实用户痛点出发推导核心功能、梳理完整用户旅程、定义可量化的成功指标；具备专业的技术方案评审能力，能够从性能、成本、可维护性、扩展性等多个维度对比不同技术路线的 trade-off，结合产品阶段与团队资源做出最优选型决策
- **掌握桌面端项目全流程初始化：**熟练完成 Electron Forge 脚手架搭建、React 19 + TypeScript 技术栈整合、Vite 多进程构建配置、TailwindCSS 4.0 样式体系接入等全链路操作，能够独立解决初始化过程中的环境依赖、路径配置、进程通信等常见问题，最终产出可正常启动的项目骨架
- **透彻理解智能体与自动分镜核心原理：**掌握 AI 剪辑智能体的节点化编排逻辑、全链路数据流、人机交互中断机制，掌握自动分镜的结构化生成原理、核心技术挑战与主流实现方案，能够清晰描述从用户自然语言输入到标准化视频项目生成的完整业务流转路径
- **建立一人全栈 AI 化研发体系认知：**深入理解 AI 时代一人全栈研发的完整工作范式，掌握从需求发散到规格落地、从架构设计到代码实现、从测试验证到发布上线的全流程 AI 提效方法，能够结合自身技术栈与工作习惯，打造适配个人研发节奏的 AI 辅助工具体系与 workflow
- **掌握 Monorepo 全栈项目架构设计：**理解 Packages 与 Apps 的职责划分，能够完成从 workflow 编排到执行引擎底层的全流程分析与设计，具备技术选型决策与权衡分析能力，能够针对应用形态、智能体框架、包管理方案等核心技术方向，从短期落地成本、长期演进空间、团队学习成本、生态成熟度、风险可控性等多个维度进行综合评估

## AI 引擎核心与算法实现

- **深入理解 LangGraph.js 图状态机核心思想：**熟练掌握 State、Node、Edge 三大核心概念与使用方法，能够从零搭建包含多业务节点的智能体 workflow，实现状态的自动流转、合并与基础的中断恢复能力，建立 LangGraph 智能体的架构设计思维，能够根据业务复杂度合理设计状态模型、拆分节点职责、编排流程走向

- **掌握智能体执行流程与 10 节点流水线**：深入理解从扫描素材、分析素材、生成创意简报、规划分镜、分镜审批中断、匹配素材、合成语音、组装时间线、验证项目到保存项目的完整工作流，熟练运用 Checkpoint 机制实现断点续跑、历史回溯与人机交互，具备复杂智能体工作流的架构设计与性能优化能力
- **掌握自动分镜核心技术**：理解从语义理解、时长划分、画面描述到转场节奏的完整技术链路，掌握「Zod Schema + Function Calling」的技术方案，能够设计受控的结构化文本生成任务，保障生产环境下 AI 输出的可靠性
- **透彻掌握 AI 应用结构化输出的完整设计方法论与稳定性保障体系**：能够从业务场景出发设计合理的数据结构，构建「Schema 定义 → 模型约束 → 运行时校验 → 失败重试 → 降级兜底」的全链路结构化输出方案

## 可视化编辑器开发

- **基于 React 19 构建三大核心页面**：掌握创作页、项目列表页、编辑器页的完整设计与实现，遵循「先搭骨架再填内容、先静态后交互、先 Mock 后真实数据」的 AI 页面开发最佳实践
- **系统掌握 AI 辅助 UI 设计方法与评审体系**：深入理解设计工具原生 AI 化的行业趋势，熟练运用 Figma AI、Pencil 两类主流工具，掌握从需求草图、线框原型、高保真设计到开发交付的完整 AI 辅助设计流程，建立专业的 UI 评审思维，能够从业务合理性、一致性、可用性、工程可实现性、可访问性五个维度对 AI 生成的界面进行系统性评审
- **掌握编辑器五区布局设计**：顶部工具栏、左侧分镜与素材面板、中间预览区、右侧属性面板、底部时间线的经典专业剪辑软件布局，理解预览区和时间线联动、时间线缩放、属性面板和选中状态联动等核心交互
- **掌握子智能体协作的开发模式**：深入理解子智能体协作的核心理念与四种经典协作模式（顺序协作、并行协作、评审协作、迭代协作），能够根据任务特点设计合理的子智能体角色划分与协作流程，明确主智能体与各子智能体的职责边界、接口定义与验收标准
- **掌握界面状态管理与事件系统**：理解智能体运行事件分发、序号防丢事件上报、AbortController 全链路取消方案，能够设计前端实时进度展示与失败自动回滚机制

## 全栈服务与 API 化

- **熟练掌握 Electron 进程模型与 IPC 通信架构**：理解主进程、渲染进程、预加载脚本的三层架构，掌握请求响应模式（invoke/handle）与事件发布订阅模式（send/on）两种 IPC 通信方式，遵循通道命名规范化、统一类型定义、标准化错误处理、最小权限原则等工程最佳实践
- **掌握 SSE（Server-Sent Events）技术**：实现 AI 生成内容的流式即时响应，支持打字机效果的实时 Token 推送，优化用户体验

- **掌握本地文件系统访问方案**：能够独立设计并实现项目文件的读写、存储与版本管理体系，熟练处理跨平台路径兼容与进程间通信安全，遵循最小权限原则完成桌面端本地存储能力的落地
- **掌握完整素材扫描与索引能力**：基于 ffprobe 实现音视频元数据的精准提取，掌握关键帧生成与素材语义预处理流程，能够处理文件损坏、格式不兼容等异常场景，完成可复用的素材管理基础模块开发

## 工程化与运维

- **掌握 pnpm Monorepo 架构管理**：理解 `apps/` 应用层与 `packages/` 核心包层的职责划分，包间依赖遵循严格的单向原则，掌握 workspace 配置与依赖统一管理
- **掌握 Provider 适配器模式的设计思想**：能够针对 LLM、TTS 等 AI 能力抽象出与厂商无关的统一接口，实现多供应商、多模型的无缝切换、故障降级与智能调度，提升系统的可扩展性与容灾能力
- **掌握工程 DSL 数据协议设计的方法论体系**：跳出单纯的 API 定义层面，从系统架构视角理解数据协议的核心价值，掌握数据协议前置设计的工程思想，能够针对复杂业务场景设计可演进、可扩展、向后兼容的数据模型，具备版本管理与数据迁移的设计思维
- **掌握 Electron Forge 跨平台打包流程**：能够正确打包二进制依赖（FFmpeg）、静态资源，处理开发环境和生产环境的路径差异，生成 Windows/macOS/Linux 三平台可运行的安装包
- **形成桌面端 AI 应用的工程化设计体系**：能够在安全边界、性能优化、代码分层、错误处理、可维护性等维度做出合理的架构权衡，建立从需求分析到方案落地的完整工程化思考框架

## 学习产物

### AI 核心引擎库 (@miaoma/video-agent)

- 基于 LangGraph.js 的完整智能体编排系统，包含 10 节点工作流流水线：扫描素材、分析素材、生成创意简报、规划分镜、分镜审批中断、匹配素材、合成语音、组装时间线、验证项目、保存项目
- 完整的图状态机实现，支持状态自动流转、合并、Checkpoint 持久化断点续跑、人机交互中断恢复
- Provider 适配器体系，支持字节火山引擎等多供应商 LLM 与 TTS 无缝切换
- 事件系统与进度上报，带序号防丢事件上报机制，支持前端实时进度展示

### 数据模型 DSL 库 (@miaoma/video-project)

- 基于 Zod 的完整视频项目数据模型，包含判别联合类型设计（VideoClip、VoiceClip、SubtitleClip、MusicClip）
- 分层设计：基础原子类型、基础资产层、片段层、分镜层、项目层
- 版本字段与向前兼容设计（schemaVersion: '1.0.0'）
- 自定义跨字段校验，包含资产存在性、时间合法性、轨道类型匹配等业务逻辑校验

## 可视化 workflow 桌面应用（Electron App）

- 基于 Electron Forge + React 19 + TypeScript + Vite + TailwindCSS 4.0 的完整桌面应用
- 三大核心页面：
  - 创作页：创意输入、素材上传、配置选项、预览示例
  - 项目列表页：项目卡片网格、搜索筛选、批量操作、空状态引导
  - 编辑器页：五区布局（顶部工具栏、左侧分镜与素材、中间预览、右侧属性、底部时间线）
- 完整 IPC 通信架构，预加载脚本安全桥接，窗口拖拽区域支持

## 本地素材扫描与索引系统

- 基于 ffprobe 的音视频元数据精准提取
- FFmpeg 关键帧生成与缩略图管理
- 素材语义预处理与索引存储
- 文件损坏、格式不兼容等异常场景处理

## TTS 文本转语音系统

- 字节火山引擎 WebSocket 实时流式合成接入
- 本地开源 TTS 模型预留扩展（Index-TTS 2 等）
- 在线/本地 TTS 服务自动路由与故障降级
- 音频缓存、文件校验、原子化音色索引存储
- 自定义音色库功能，支持声音克隆

## FFmpeg 视频渲染与烧录系统

- 复杂滤镜图构建，支持多轨道视频合成、音频混音、字幕烧录

- H.264 编码参数调优（CRF、preset），画质与体积平衡
- 跨平台路径、字体兼容、时间轴对齐等工程问题处理
- Electron 自定义协议安全机制，私有协议加载本地媒体文件，Range 请求支持视频进度拖动

## 技术选型

### 整体架构分层设计

本项目采用经典的**四层分层架构**，每层职责清晰，依赖方向明确，符合单一职责原则：

| 分层   | 职责范围               | 核心技术选型                                                | 设计思想                       |
|------|--------------------|-------------------------------------------------------|----------------------------|
| 界面层  | 用户交互、可视化编辑、状态展示    | React 19 +<br>TailwindCSS 4.0 +<br>Shadcn/ui + XYFlow | 组件化开发，单向数据流，分离交互与业务逻辑      |
| 智能体层 | AI 工作流编排、状态管理、人机协作 | LangGraph.js +<br>LangChain.js + Zod                  | 图状编排，有状态执行，原生支持中断恢复        |
| 音视频层 | 媒体处理、元数据提取、导出合成    | FFmpeg/ffprobe +<br>Index-TTS 2 + 火山 TTS              | 模块化抽象，混合云架构，统一 Provider 接口 |
| 桌面底层 | 系统能力封装、安全边界、打包分发   | Electron + Electron<br>Forge + pnpm                   | 遵循 Electron 安全最佳实践，最小权限原则  |

### 详细技术选型说明

#### 工程与包管理

- **包管理器:** `pnpm` + workspace
  - 优势：硬链接共享依赖，节省磁盘空间，安装速度快，Monorepo 原生支持良好
  - 适用场景：多包项目，清晰划分模块依赖
- **Monorepo 架构:** pnpm workspace
  - 为什么选 Monorepo：核心引擎和应用放在同一个仓库，版本对齐方便，跨模块修改一次性提交，文档代码同步
  - 分层原则：`packages` 放可复用核心库，`apps` 放应用入口，依赖从下往上，不循环依赖

## 桌面端框架

- **应用框架:** Electron
  - 对比选型:
    - **vs Tauri:** Electron 生态更成熟，开发资料多，打包稳定，对 Node.js 原生模块支持更好；Tauri 体积更小但对原生 Node 模块集成不如 Electron 友好，本项目依赖 FFmpeg 二进制，不需要极致体积，所以选 Electron
    - **vs 原生开发:** Electron 复用前端技术栈，一人全栈可以搞定，开发效率高，满足当前项目阶段需求
  - 安全模型：主进程拥有系统权限，渲染进程沙箱隔离，预加载脚本桥接受控 API，遵循最小权限原则
- **构建工具:** Vite (渲染进程) + Electron Forge (打包分发)
  - Vite 热更新快，开发体验好
  - Electron Forge 一站式处理打包、签名、公证、发布，生态完善，减少配置工作量

## 前端与 UI

- **前端框架:** React 19 + TypeScript
  - React 19 带来最新的并发特性、useOptimistic 等 Hooks，适合局部重生成乐观更新场景
  - TypeScript 提供静态类型检查，配合 Zod 做运行时校验，双重保障类型安全
- **样式体系:** TailwindCSS 4.0 + Shadcn/ui
  - Tailwind 4 采用 CSS-first 配置，编译更快，体积更小，原子类 CSS 开发效率高
  - Shadcn/ui 提供高质量 unstyled 基础组件，灵活定制，符合项目设计风格
- **可视化编辑:** 专业剪辑使用经典五区布局，核心交互不依赖额外可视化库；如果需要工作流可视化编排，可集成 XYFlow (React Flow)

## AI 与智能体

- **智能体编排:** LangGraph.js
  - 核心优势：原生支持有状态图执行、Checkpoint 持久化、人机中断，完美匹配本项目「分镜审批需要用户确认后继续」场景
  - 对比手写状态机：LangGraph 已经处理了状态合并、持久化、中断恢复这些通用问题，我们只需要关注业务节点，开发效率更高

- **AI 基础层:** `LangChain.js`
  - 提供模型抽象、Prompt 模板、工具调用等基础能力，和 LangGraph 无缝集成
- **大模型服务:** 字节火山方舟
  - 兼容 OpenAI 接口协议，中文模型效果优秀，接入简单，国内访问速度快
- **数据校验与结构化:** `Zod`
  - **一处定义，多处复用:** Schema 定义一次，同时满足 TypeScript 类型推导、运行时校验、AI 结构化输出约束，完美匹配 AI 项目需求
- **结构化输出方案:** `Function Calling` + `Zod` + 自动重试
  - 为什么选这个方案：相比纯 Prompt 约束，Function Calling 格式稳定性高很多；Zod 做最后一层兜底校验，失败自动重试，生产级可靠性有保障

## 音视频与 TTS

- **媒体处理:** `FFmpeg` + `ffprobe`
  - 工业界几十年验证，支持几乎所有格式，功能全面，性能优秀，业界所有主流剪辑工具底层都用它，生态成熟不用重复造轮子
- **TTS 架构:** 混合云方案
  - **在线 TTS:** 字节火山引擎
    - 优势：中文效果顶尖，音色丰富，情感细腻，接入简单，不需要本地算力
  - **本地 TTS:** `Index-TTS 2`
    - 优势：支持零样本音色克隆，模型体积小，推理速度快，中文效果在开源模型中属于第一梯队，满足用户自定义音色和隐私需求
  - **架构设计:** 抽象统一 `TtsProvider` 接口，适配器模式实现，上层业务无感知切换，支持故障降级

## 存储

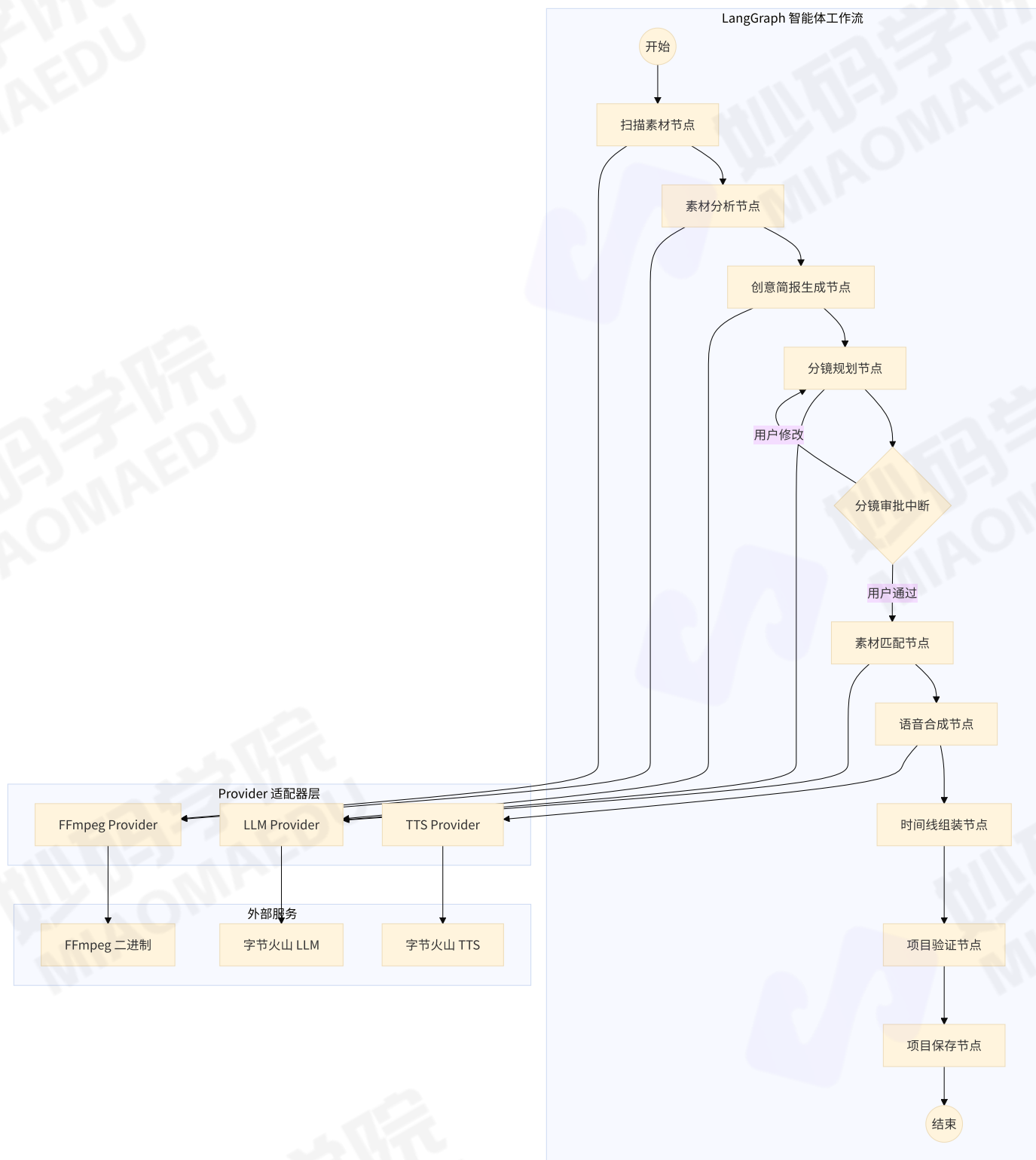
- **项目文件:** JSON 格式 + 分层目录结构
  - 调试友好，Git 可对比，不需要特殊数据库，简单够用
  - 目录结构分离： `project.json` + `assets/` + `cache/` + `exports/`，职责清晰，缓存可单独清理
- **素材索引:** `SQLite`
  - 支持高效查询、增量更新、事务，单文件存储，迁移方便，适合本地桌面应用

## 项目架构

### @miaoma/video-agent（智能体核心引擎）

**简介：**项目的核心业务逻辑库，负责 LangGraph 智能体编排、状态管理、事件系统、Provider 适配器。

- **LangGraph.js**：图状态机框架，提供 State、Node、Edge 核心抽象，内置 Checkpoint 持久化、中断恢复、人机交互能力
- **LangChain.js**（@langchain/core、@langchain/ollama）：提供模型抽象、Prompt 模板、工具调用基础能力
- **Zod**：定义数据结构 Schema 与运行时校验，作为 AI 结构化输出的约束标准
- **字节火山引擎 SDK**：LLM 推理与 TTS 语音合成的官方供应商接入
- **ffmpeg**：Node.js FFmpeg 封装，用于媒体探针与关键帧提取



## @miaoma/video-project (数据模型 DSL)

**简介：**项目的统一数据协议标准，一处定义多处复用，同时作为 TypeScript 类型、运行时校验、AI 输出约束、自动文档来源。

- **Zod：**核心 Schema 定义库，提供类型推导、运行时校验、组合嵌套能力

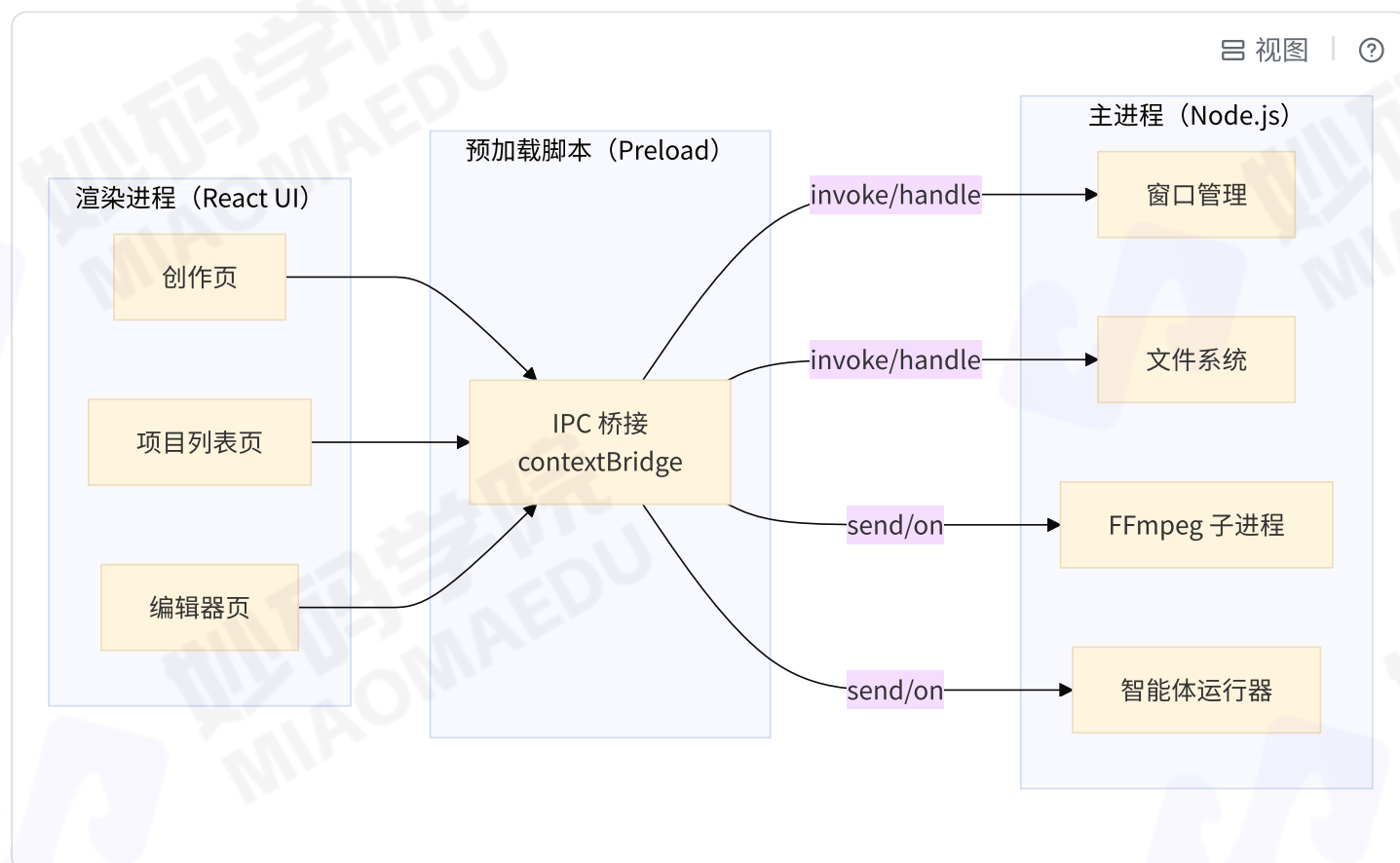
- **判别联合模式**：通过 `kind` 字段区分不同类型的 TimelineClip
- **版本管理**：schemaVersion 字段支持向前兼容与数据迁移

## apps/desktop (Electron 桌面应用)

**简介**：基于 Electron 的桌面端应用，采用三层进程架构，提供完整的可视化编辑体验。

- **Electron 38**：桌面端框架，主进程 + 渲染进程 + 预加载脚本三层模型
- **Electron Forge**：脚手架、构建、打包、发布全流程工具
- **React 19**：最新的 React 版本，利用新的 Hooks 和并发特性
- **TypeScript 5**：类型安全保障
- **Vite**：多进程构建配置，极速开发体验
- **TailwindCSS 4.0**：CSS-first 的样式框架，零运行时
- **Shadcn/ui**：组件库，提供一致的 UI 风格
- **Zustand**（可选）：轻量级状态管理，管理编辑器复杂状态

### 三层进程架构



### 基础设施与工程化

- **pnpm**: Monorepo 包管理, workspace 配置
- **Husky + Commitlint**: 代码规范与提交检查
- **FFmpeg**: 音视频编解码与渲染, 随应用分发二进制文件
- **ffprobe**: 媒体元数据探针, 用于素材分析
- **lowdb** (可选): 轻量级 JSON 数据库, 用于项目元数据存储

## 简历描述

### 初级 / 中级

#### AI 智能剪辑桌面应用前端开发

- **技术栈**: TypeScript, React 19, Electron, TailwindCSS 4.0, Zod, LangGraph.js, FFmpeg
- **项目简介**: 独立开发了一款桌面端 AI 智能剪辑工具, 用户输入自然语言描述即可自动生成完整短视频, 支持本地素材管理、自定义音色克隆、成品视频导出, 满足隐私敏感场景本地化需求, 相比传统剪辑效率提升 5 倍以上。
- **工作内容与成果**:
  - 基于 React 19 + TailwindCSS 完成创作页、项目列表页、编辑器页三大核心页面开发, 实现专业剪辑经典五区布局, 支持项目新建、打开、删除、重命名完整流程, 完整处理空状态、加载、错误等边界场景, 提升用户体验。
  - 基于 ffprobe 实现本地素材扫描与元数据提取功能, 设计增量扫描算法, 通过文件修改时间+大小对比跳过未修改文件, 将二次扫描耗时从分钟级压缩到秒级; 完整处理文件损坏、格式不兼容、权限不足等异常场景, 单个文件失败不影响整体扫描。
  - 接入字节火山 TTS 服务, 基于 WebSocket 流式协议实现语音合成, 设计语音缓存层, 相同文本+参数直接返回缓存, 避免重复调用, 大幅降低 API 成本提升响应速度; 合成后自动调用 ffprobe 探测真实音频时长, 用于分镜时间线精准对齐。
  - 基于 Electron Forge 完成跨平台打包配置, 解决 FFmpeg 二进制依赖打包路径问题, 正确处理开发环境和生产环境路径差异, 成功输出 Windows/macOS 三平台可运行安装包。

### 中级 / 高级

- **技术栈:** TypeScript, Electron, React 19, LangGraph.js, FFmpeg, Zod, Electron Forge, pnpm
- **项目简介:** 设计并开发了一款企业级桌面端 AI 智能剪辑产品，是面向音视频剪辑场景的**垂类智能体**，采用混合云架构，支持自然语言输入一键生成完整短视频，支持本地零样本音色克隆，完整覆盖从创意输入到成片导出的全流程，帮助创作者将制作周期从小时级缩短到分钟级。
- **工作内容与成果:**
  - **架构设计:** 主导整体架构设计，采用 pnpm Monorepo 分层架构，清晰划分「界面层/智能体层/音视频层/桌面底层」四层职责，基于 Zod 实现全链路数据校验，从输入到输出每一层都有格式保障，**有效规避 LLM 输出不稳定问题，将线上格式错误率降低 90% 以上。**
  - **智能体设计:** 基于 LangGraph.js 设计实现了完整的剪辑智能体工作流，包含素材扫描、创意生成、分镜规划、分镜审批中断、素材匹配、语音合成、时间线组装全链路；利用 Checkpoint 持久化机制实现断点续跑，支持用户修改分镜后从中断处继续执行，保留所有中间状态，**提升人机协作体验。**
  - **音视频导出:** 设计实现了类型安全的 FFmpeg 命令构建器，将多轨道时间线自动转换为 `filter\_complex` 复杂滤镜图，系统化解决了 atempo 变速串联、Windows 路径转义、字幕字体跨平台兼容、PTS 时间戳对齐等常见工程坑点；实现了 stderr 进度解析和 AbortController 全链路取消，前端实时展示百分比和剩余时间，支持用户随时终止并自动清理临时文件，**不会残留僵尸进程。**
  - **TTS 架构设计:** 设计了「在线+本地」混合云 TTS 架构，抽象出与厂商无关的统一 Provider 接口，默认走火山在线 TTS 保证音质，自定义音色走本地 Index-TTS 2 保护用户隐私；实现了基于内容哈希的语音缓存和 LRU 淘汰，相同内容不重复调用，**大幅降低 API 调用成本，提升重复场景响应速度 10 倍以上。**
  - **安全与分发:** 设计了 Electron 自定义协议安全加载本地媒体，彻底解决 `file://` 协议带来的跨域和权限问题，完整实现 Range 请求协议支持播放器进度拖动；掌握了 macOS Developer ID 签名+公证完整流程和 Windows 代码签名最佳实践，能够产出符合系统安全要求的安装包，**避免用户打开时遇到未知开发者安全警告。**
  - **工程化实践:** 在项目中实践了 AI 时代的混合开发流程：探索阶段使用 Vibe Coding 快速验证方向，稳定阶段切换到 Spec Driven 开发保证质量，**平衡探索效率和工程质量，充分发挥 AI 提效优势同时控制技术债务。**

## 课程内容

### 1. 需求分析、项目架构设计智能体执行流程与 AI Coding 实践

## 初中级

1. **深入理解 AI 开发方法论体系：**系统掌握 Vibe Coding 迭代式开发、Spec 驱动的规格化研发、Skills 技能体系化赋能、MCP 工具互联协议等 AI 时代核心开发范式，清晰理解每种方法的核心思想、落地原则与适用边界，能够准确判断不同场景下的最优开发模式，避免方法错配导致的效率损耗。
2. **掌握桌面端项目全流程初始化：**熟练完成 Electron Forge 脚手架搭建、React 19 + TypeScript 技术栈整合、Vite 多进程构建配置、TailwindCSS 4.0 样式体系接入等全链路操作，能够独立解决初始化过程中的环境依赖、路径配置、进程通信等常见问题，最终产出可正常启动的项目骨架。
3. **吃透智能体与自动分镜核心原理：**透彻理解 AI 剪辑智能体的节点化编排逻辑、全链路数据流、人机交互中断机制，掌握自动分镜的结构化生成原理、核心技术挑战与主流实现方案，能够清晰描述从用户自然语言输入到标准化视频项目生成的完整业务流转路径。
4. **具备独立搭建与验证能力：**能够独立完成 pnpm Monorepo 工作区配置、Electron 主进程与渲染进程的 IPC 通信联调、基础 UI 页面的渲染验证，最终交付一个可正常启动、可完成基础交互、结构清晰可扩展的桌面端应用原型，为后续功能模块开发打下稳固的工程基础。

## 高级

1. **掌握产品需求分析与技术评审方法论：**系统掌握工具类产品的需求拆解与分析方法，能够从真实用户痛点出发推导核心功能、梳理完整用户旅程、定义可量化的成功指标；具备专业的技术方案评审能力，能够从性能、成本、可维护性、扩展性等多个维度对比不同技术路线的 trade-off，结合产品阶段与团队资源做出最优选型决策。
2. **建立一人全栈 AI 化研发体系认知：**深入理解 AI 时代一人全栈研发的完整工作范式，掌握从需求发散到规格落地、从架构设计到代码实现、从测试验证到发布上线的全流程 AI 提效方法，能够结合自身技术栈与工作习惯，打造适配个人研发节奏的 AI 辅助工具体系与 workflow。
3. **具备技术选型决策与权衡分析能力：**具备成熟的架构师选型思维，能够针对应用形态、智能体框架、包管理方案等核心技术方向，从短期落地成本、长期演进空间、团队学习成本、生态成熟度、风险可控性等多个维度进行综合评估，清晰阐述每个方案的收益与代价，输出结构化的选型决策报告。
4. **构建 AI 时代全栈开发思维体系：**跳出传统前端 / 后端的角色边界，建立 AI 时代下的全栈开发思维体系，理解「大模型作为新型编程接口」「自然语言作为新型编程语言」的核心逻辑，掌握人机协作的开发新范式，能够借助 AI 工具快速拓展自身能力边界，实现从「写代码」到「做产品」的能力升级。

## 2. UI 设计评审、基于 spec 编程子智能体执行与基础框架实现

## 初中级

1. **系统掌握 AI 辅助 UI 设计方法与评审体系**：深入理解设计工具原生 AI 化的行业趋势，熟练运用 Figma AI、Pencil 两类主流工具，掌握从需求草图、线框原型、高保真设计到开发交付的完整 AI 辅助设计流程；建立专业的 UI 评审思维，能够从业务合理性、一致性、可用性、工程可实现性、可访问性五个维度对 AI 生成的界面进行系统性评审，输出结构化的改进建议。
2. **熟练掌握 Spec 驱动的需求拆解与任务规划方法**：透彻理解 Spec 驱动开发的核心理念，能够将一份完整的产品规格说明书，按照单一职责、大小适中、可独立交付、依赖清晰的原则，拆解为可执行的开发任务清单；掌握任务依赖图的构建方法，能够识别关键路径，合理估算工作量，制定科学的项目排期与风险应对方案。
3. **具备 Zod 数据模型 DSL 设计与实现能力**：深入理解 Schema 驱动开发的工程价值，熟练掌握 Zod 的核心 API 与高级技巧，能够独立设计复杂业务场景的数据模型 Schema，灵活运用对象、数组、判别联合、自定义校验等能力；理解 DSL 在工程化中的作用，能够从类型安全、运行时校验、自动文档三个维度设计高质量的数据协议。
4. **能够独立完成三大核心页面的基础开发**：能够基于 AI 辅助开发模式，独立完成创作页、项目列表页、编辑器页三大核心页面的基础 UI 开发，掌握「先搭骨架再填内容、先静态后交互、先 Mock 后真实数据」的 AI 页面开发最佳实践，最终交付结构清晰、样式规范、交互完整的页面原型。

## 高级

1. **掌握子智能体协作的开发模式与最佳实践**：深入理解子智能体协作的核心理念与四种经典协作模式，能够根据任务特点设计合理的子智能体角色划分与协作流程；掌握子智能体团队的设计方法，明确主智能体与各子智能体的职责边界、接口定义与验收标准；能够识别子智能体协作的适用边界与局限性，在实际项目中合理运用，最大化协作效率。
2. **建立工程 DSL 数据协议设计的方法论体系**：跳出单纯的 API 定义层面，从系统架构视角理解数据协议的核心价值，掌握数据协议前置设计的工程思想；能够针对复杂业务场景设计可演进、可扩展、向后兼容的数据模型，具备版本管理与数据迁移的设计思维；深入理解判别联合、资产与片段分离等高级设计模式，能够在架构层面做出合理的技术决策。
3. **具备 UI 设计评审与产品体验优化能力**：建立专业的产品视角与设计审美，能够站在用户体验高度对 UI 方案进行评审，不仅发现表面问题，更能挖掘深层的体验缺陷；掌握专业的评审方法论与沟通技巧，能够输出有建设性的、可落地的改进建议；理解智能剪辑产品与传统剪辑产品的体验差异，能够在 AI 产品的交互设计上提出创新思路。

4. **构建 Spec 驱动开发的工程化思维体系**：深入理解 Spec 在 AI 开发中的核心地位，建立「规格先行、代码随后」的工程化思维；掌握从需求到 Spec、从 Spec 到任务、从任务到代码的完整链路方法论，能够将 Spec 作为团队协作的统一语言与唯一真相来源；理解 Spec 驱动与 Vibe Coding 的辩证关系，能够在项目不同阶段灵活切换两种模式，平衡探索效率与工程质量。

### 3. 界面主体、素材扫描索引、基于 Langgraph.js 模型接入及文本转语音 (TTS) 实现

#### 初中级

1. **系统掌握 Electron 桌面应用的本地文件系统访问方案**：能够独立设计并实现项目文件的读写、存储与版本管理体系，熟练处理跨平台路径兼容与进程间通信安全，遵循最小权限原则完成桌面端本地存储能力的落地
2. **能够独立搭建完整的本地素材扫描与索引能力**：基于 ffprobe 实现音视频元数据的精准提取，掌握关键帧生成与素材语义预处理流程，能够处理文件损坏、格式不兼容等异常场景，完成可复用的素材管理基础模块开发
3. **深入理解 LangGraph.js 的图状态机核心思想**：熟练掌握 State、Node、Edge 三大核心概念与使用方法，能够从零搭建包含多业务节点的智能体工作流，实现状态的自动流转、合并与基础的中断恢复能力
4. **独立完成字节火山引擎大语言模型与语音合成服务的接入落地**：掌握 API 调用、错误处理与基础重试机制，能够实现符合业务要求的结构化输出与语音文件生成，完成 AI 能力与桌面应用的基础整合

#### 高级

1. **透彻掌握 AI 应用结构化输出的完整设计方法论与稳定性保障体系**：能够从业务场景出发设计合理的数据结构，构建「Schema 定义 → 模型约束 → 运行时校验 → 失败重试 → 降级兜底」的全链路结构化输出方案，保障生产环境下 AI 输出的可靠性
2. **建立 LangGraph 智能体的架构设计思维**：能够根据业务复杂度合理设计状态模型、拆分节点职责、编排流程走向，熟练运用 Checkpoint 机制实现断点续跑、历史回溯与人机交互，具备复杂智能体工作流的架构设计与性能优化能力
3. **熟练掌握 Provider 适配器模式的设计思想**：能够针对 LLM、TTS 等 AI 能力抽象出与厂商无关的统一接口，实现多供应商、多模型的无缝切换、故障降级与智能调度，提升系统的可扩展性与容灾能力

4. **形成桌面端 AI 应用的工程化设计体系**：能够在安全边界、性能优化、代码分层、错误处理、可维护性等维度做出合理的架构权衡，建立从需求分析到方案落地的完整工程化思考框架

## 4. 成片全链路联调、基于 FFmpeg 视频烧录、多端应用打包及实战总结高分面试导引

### 初中级

1. **深入掌握 FFmpeg 复杂滤镜图的构建原理**：能够独立实现多轨道视频合成、音频混音、字幕烧录等专业剪辑功能，理解 H.264 编码参数（CRF、preset）对画质和体积的影响，能够处理跨平台路径、字体兼容、时间轴对齐等工程问题
2. **完整实现自定义音色库功能**：掌握本地开源 TTS 模型（Index-TTS 2）的接入方法，理解声音克隆的基本原理，能够实现在线/本地 TTS 服务的自动路由，支持用户上传参考音频创建专属音色
3. **深入理解 Electron 自定义协议的安全机制**：能够注册安全的私有协议加载本地媒体文件，解决 file:// 协议的跨域和权限问题，支持 Range 请求实现视频进度拖动，掌握最小权限原则下的文件访问控制
4. **熟练掌握 Electron Forge 跨平台打包流程**：能够正确打包二进制依赖（FFmpeg）、静态资源，处理开发环境和生产环境的路径差异，生成 Windows/macOS/Linux 三平台可运行的安装包

### 高级

1. **精通 LangGraph 智能体长流程架构设计与容错体系**：掌握多节点 AI 剪辑任务的增量状态合并、Checkpoint 持久化断点续跑、人机交互中断机制落地；可独立设计任务事件分发、AbortController 全链路取消方案，解决长时 AI 任务卡死、状态错乱、资源泄露问题，搭建带序号防丢事件上报系统，实现前端实时进度展示、失败自动回滚。
2. **掌握混合云 TTS 架构与 AI 音色工程化优化**：设计在线火山 TTS 与本地 Index-TTS2 双路由降级架构，搭建音频缓存、文件校验、原子化音色索引存储体系；解决本地模型服务异常、参考音频音质降噪、多倍速 atempo 滤镜串联适配等生产问题，实现音色导入、预览、删除全链路安全管控。
3. **具备 AI 剪辑产品架构设计与面试项目包装能力**：能够自顶向下完成 Monorepo 分层架构拆分（UI 层 / 智能体层 / 音视频层 / 桌面底层），搭建 Zod 全链路数据校验体系规避 LLM 输出不稳定；掌握项目 STAR 描述、技术难点量化成果输出、高频面试底层原理应答思路，可独立产出可用于求职的完整实战项目，具备需求拆解、技术选型、线上问题复盘能力。

4. 掌握产品进阶扩展方案设计：能够独立规划素材检索、多模态素材解析、AI 智能转场、云端协同版本管理等高阶功能迭代方案，针对短视频、电商、自媒体不同业务场景做音视频、AI 能力定制化适配。

代码 commit 截图

所有分支

显示远程分支

层级顺序

扩展视图

跳转到

| 图表 | 描述                                                                                                                        | 提交      | 作者                       | 日期               |
|----|---------------------------------------------------------------------------------------------------------------------------|---------|--------------------------|------------------|
|    | <a href="#">main</a> <a href="#">origin/main</a> docs: codex chat history and fixed node version to solve electron unp... | a7042ee | Heyi <miaomaedu@163.com> | 2026年7月12日 08:45 |
|    | feat: expansion points include a voice library and local voice tts generation                                             | 1fc54e8 | Heyi <miaomaedu@163.com> | 2026年6月30日 03:21 |
|    | feat: multi-platform packaging and distribution                                                                           | f75240d | Heyi <miaomaedu@163.com> | 2026年6月30日 02:42 |
|    | feat: export the final video using FFmpeg and support multi-platform rendering                                            | 64ef584 | Heyi <miaomaedu@163.com> | 2026年6月30日 02:03 |
|    | feat: add song config and preview, supports turning subtitles and songs on and off                                        | 2d0db52 | Heyi <miaomaedu@163.com> | 2026年6月30日 01:06 |
|    | feat: subtitle style setting                                                                                              | 4d8eb91 | Heyi <miaomaedu@163.com> | 2026年6月30日 00:16 |
|    | feat: regenerate audio using different tones                                                                              | 6359a04 | Heyi <miaomaedu@163.com> | 2026年6月29日 23:23 |
|    | feat: text to regenerates content                                                                                         | 4fd9971 | Heyi <miaomaedu@163.com> | 2026年6月29日 03:04 |
|    | feat: frames panel render creation message list                                                                           | dcf7548 | Heyi <miaomaedu@163.com> | 2026年6月29日 02:20 |
|    | feat: video creation agents execute complete chain and interaction implementation                                         | 81bf4a1 | Heyi <miaomaedu@163.com> | 2026年6月29日 01:33 |
|    | feat: seek preview, left panel scene sync                                                                                 | c3c45cd | Heyi <miaomaedu@163.com> | 2026年6月29日 00:45 |
|    | feat: assets composite and base previewer finished                                                                        | 8ec1976 | Heyi <miaomaedu@163.com> | 2026年6月29日 00:01 |
|    | feat: projects list and remove, project title editable                                                                    | 2fc39c7 | Heyi <miaomaedu@163.com> | 2026年6月28日 03:23 |
|    | feat: agent flow invoke and progress show, finished core flow                                                             | fe3aef4 | Heyi <miaomaedu@163.com> | 2026年6月28日 00:27 |
|    | feat: video project data to editor json                                                                                   | a3b4139 | Heyi <miaomaedu@163.com> | 2026年6月27日 01:38 |

已依照路径排序

.gitignore

packages/video-project/package.json

packages/video-project/tsconfig.json

packages/video-project/tdown.config.ts

packages/video-project/src/index.ts

feat: add video project schema package

提交: 26aa9ddcd5a43835c21f21cf89ca0670aa7b81a4 [26aa9dc]

父级: 44f0cc9b9f

作者: Heyi <miaomaedu@163.com>

日期: 2026年6月23日 GMT+8 03:40:19

.gitignore

块 1: 行 46-52

46 46 # Output coverage \*.lcov

47 47

48 48

49 49 + \*\*/dist/

49 50 .next

50 51 .nuxt

51 52 .vuepress/dist

回滚区块

“企业级项目”指采用企业常用技术栈及开发流程的教学模拟项目，不等同于商业生产环境使用的项目